Project Summary and Planning Chart

Project Title	LLM-Assisted Power System Modelling with Simplus Grid Tool
Student Name	Benjamin Kahle
Student ID	40178005
Supervisor	Prof. ZHU, Yue
Programme	EXGB
Project Code	YZ-6

1. Project Aims and Objectives

Building power system simulation models requires significant manual effort in defining structures, selecting component models, and configuring parameters. The primary objective of this project is to develop an intelligent modelling interface that combines a Large Language Model (LLM) with the Simplus Grid Tool (SimplusGT) to generate simulation models directly from natural language instructions.

The scope is strictly bounded: the user provides plain language text defining the system scale, inverter type, and network structure for AC power systems only (DC and hybrid AC/DC networks are explicitly outside the scope of this project). The LLM translates this request and automatically generates the required SimplusGT configuration files. These files are then executed in MATLAB/Simulink through the SimplusGT environment.

2. System Architecture

Figure 1 illustrates the system architecture. The workflow begins with a preprocessing stage that validates the user's raw text and wraps it with strict system instructions, ensuring the LLM operates within the defined scope (e.g., AC networks only). A Python-based validation layer then acts as a bridge between the LLM API and MATLAB. The process is closed-loop: if the LLM generates an invalid JSON configuration, the validation layer catches the structural or mathematical error and feeds it back to the LLM for correction. This loop is capped at three iterations to prevent an unbounded correction loop.

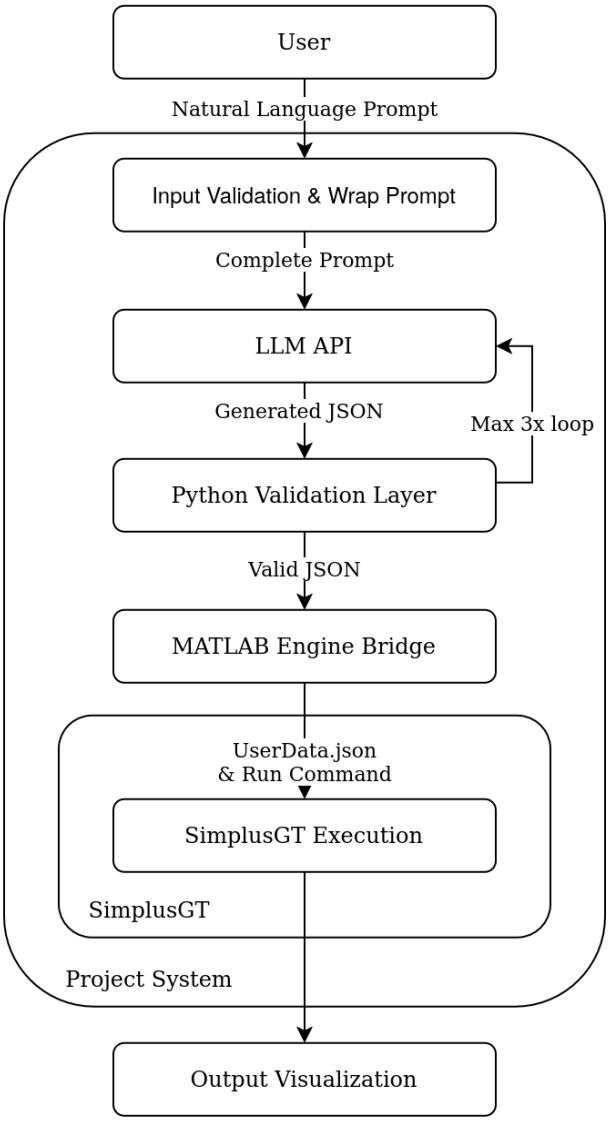


Figure 1: System Architecture and Execution Pipeline

3. Background Study and Literature Search

Preparatory work involves two distinct domains. First, power system theory must be researched to understand grid topologies and the specific parameterization required for Grid-Following (GFL) and Grid-Forming (GFM) inverters within SimplusGT. Second, contemporary LLM engineering techniques must be evaluated, specifically focusing on prompt engineering constraints and strict JSON schema generation to reliably output complex data arrays without formatting degradation.

4. Methodology and Work Approach

The core translation layer uses an LLM to parse text into strict Pydantic data schemas that map exactly to SimplusGT's *UserData.json* requirements. For execution, the system utilises the MATLAB Engine API for Python to pass the generated model files

into the SimplusGT environment and programmatically trigger *UserMain.m*.

This is a deliberate methodological choice: relying on the Python-to-MATLAB bridge centralises the error-handling logic and allows the pipeline to run headless during automated benchmarking, isolating translation errors from execution errors.

5. Implementation

The software stack consists of Python, MATLAB/Simulink, Simplus Grid Tool, and a commercial LLM API.

The main deliverables are the parsing algorithm, the mathematical validation script, and the automated execution bridge. The primary interface will be a lightweight Python CLI that leverages the native SimplusGT GUI for output visualisation. Developing a custom frontend would be an unnecessary distraction from the core translation logic; relying on the native UI provides a deterministic, easily quantifiable baseline. If the core engine stabilises early, a standalone web dashboard is proposed as an optional stretch goal.

6. Interpretation of Results

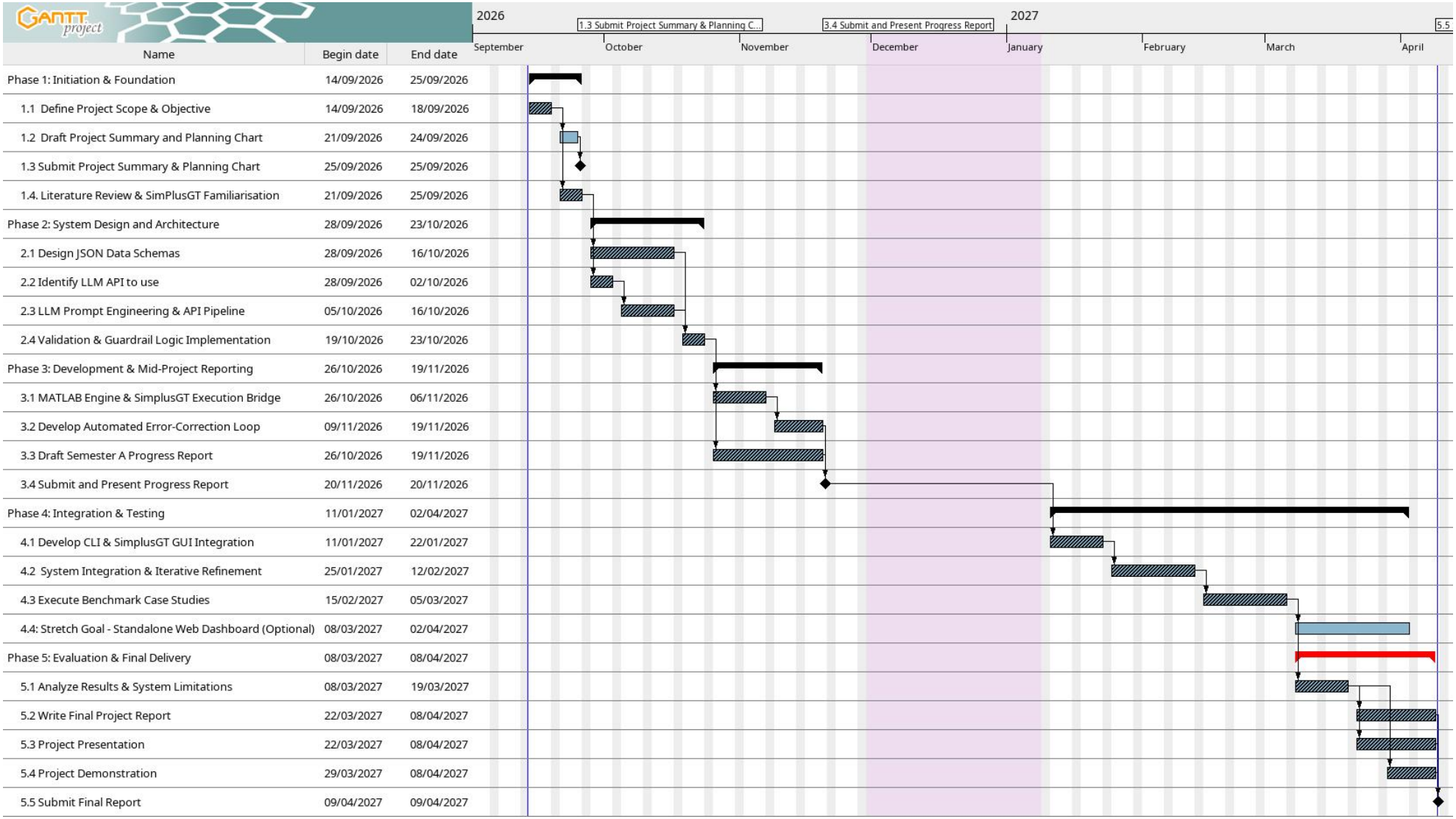
The system's viability will be characterised quantitatively. To establish a reliable, deterministic baseline, the 12 standard AC power system examples provided natively within SimplusGT will be used as benchmark case studies. Key evaluation metrics include the percentage of valid JSON schemas successfully generated against these 12 benchmarks and the rate of MATLAB executions that complete without syntax failure. A perfect generation rate is not required for project success. If benchmarking shows the LLM cannot reliably configure complex models, the evaluation will instead focus on analyzing these limitations and proposing alternative solutions or improvements.

7. Conclusion

The project's primary contribution is a prototype that bridges natural language input with rigorous grid simulation, automating the definition of AC network topologies and removing much of the manual trial-and-error in baseline parameter selection. Should benchmarking expose reliability limits on complex multi-inverter networks, these will be documented as a formal analysis of failure modes to inform future architectural or hybrid-workflow improvements.

8. Project Planning & Milestone Schedule

Figure 2 outlines the work progress across both semesters. It details the required phases, including literature search, system architecture, integration, iterative testing, and all mandatory departmental milestones and presentations. The critical path is defined by the technical pipeline, while reporting tasks are run in parallel to generate schedule slack.



Name	Begin date	End date
Phase 1: Initiation & Foundation	14/09/2026	25/09/2026
1.1 Define Project Scope & Objective	14/09/2026	18/09/2026
1.2 Draft Project Summary and Planning Chart	21/09/2026	24/09/2026
1.3 Submit Project Summary & Planning Chart	25/09/2026	25/09/2026
1.4. Literature Review & SimPlusGT Familiarisation	21/09/2026	25/09/2026
Phase 2: System Design and Architecture	28/09/2026	23/10/2026
2.1 Design JSON Data Schemas	28/09/2026	16/10/2026
2.2 Identify LLM API to use	28/09/2026	02/10/2026
2.3 LLM Prompt Engineering & API Pipeline	05/10/2026	16/10/2026
2.4 Validation & Guardrail Logic Implementation	19/10/2026	23/10/2026
Phase 3: Development & Mid-Project Reporting	26/10/2026	19/11/2026
3.1 MATLAB Engine & SimplusGT Execution Bridge	26/10/2026	06/11/2026
3.2 Develop Automated Error-Correction Loop	09/11/2026	19/11/2026
3.3 Draft Semester A Progress Report	26/10/2026	19/11/2026
3.4 Submit and Present Progress Report	20/11/2026	20/11/2026
Phase 4: Integration & Testing	11/01/2027	02/04/2027
4.1 Develop CLI & SimplusGT GUI Integration	11/01/2027	22/01/2027
4.2 System Integration & Iterative Refinement	25/01/2027	12/02/2027
4.3 Execute Benchmark Case Studies	15/02/2027	05/03/2027
4.4: Stretch Goal - Standalone Web Dashboard (Optional)	08/03/2027	02/04/2027
Phase 5: Evaluation & Final Delivery	08/03/2027	08/04/2027
5.1 Analyze Results & System Limitations	08/03/2027	19/03/2027
5.2 Write Final Project Report	22/03/2027	08/04/2027
5.3 Project Presentation	22/03/2027	08/04/2027
5.4 Project Demonstration	29/03/2027	08/04/2027
5.5 Submit Final Report	09/04/2027	09/04/2027

Figure 2: Project Gantt Chart